Centro Universitário de Excelência
Sistemas de Informação

Título da Apresentação

Autor: Joabe Irlan Almeida Lopes
Saulo de Melo Ramos

Feira de Santana, 2025

INTRODUÇÃO

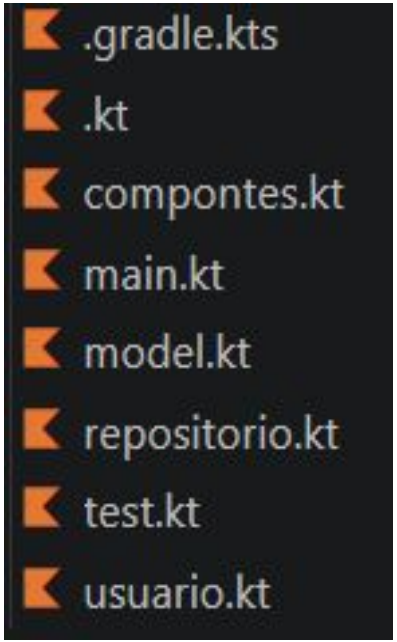
Sistema Dendê Eventos

1. **Aplicação em Kotlin via console**

2. **Gerencia usuários e eventos**

3. **Armazena dados em memória**

4. **Utiliza lógica simples e interativa**



ESTRUTURA DO PROJETO

- Uso de `data class` para modelar dados
- Uso de `enum` para padronização
- Uso de listas (`mutableListOf`)Uso de listas (`mutableListOf`)
- Entrada de dados via `Scanner`

DATA CLASS: USUÁRIO

```
data class Usuario(
    val id: Int,
    var nome: String,
    var email: String,
    var status: StatusUsuario = StatusUsuario.ATIVO
)
```

- Nome
- Email
- Senha
- Status (ativo/inativo)
- Tipo (organizador ou comum)

Permite controle de acesso e identificação

DATA CLASS: EVENTO

- Nome do evento
- Data e local
- Capacidade
- Preço
- Status (ativo/inativo)
- Organizador responsável

```
data class Evento(
    val id: Int,        You, mês passado
    var nome: String,
    var organizadorId: Int,
    var estaAtivo: Boolean = true
)
```

Base para gerenciamento de eventos

FUNÇÕES AUXILIARES

- `(readInt)` → leitura segura de números
- `(readString)` → leitura validada de texto
- `(printTable )` → exibição organizada

👉 Melhoram organização e validação

```
fun printTable(header: String, items: List<Any>) {
```

```
fun readInt(message: String, errorMessage: String, range: IntRange = 0..Int.MAX_VALUE): Int {
```

```
fun readString(message: String, errorMessage: String, minLength: Int = 0): String {
```

REPOSITÓRIO

Armazena dados em memória:

- Lista de usuários
- Lista de eventos

Funções principais:

- salvar
- buscar
- listar
- excluir

```
object Repositorio {
    val usuarios = mutableListOf<Usuario>()
    val eventos = mutableListOf<Evento>()

    fun salvarUsuario(usuario: Usuario) = usuarios.add(usuario)

    fun buscarUsuarioPorId(id: Int) = usuarios.find { it.id == id }

    fun listarAtivos() = usuarios.filter { it.status == StatusUsuario.ATIVO }

    fun excluirUsuario(id: Int) = usuarios.removeIf { it.id == id }
}
```

FUNCIONALIDADES DO SISTEMA

- cadastrar usuários
- inativar/reativar contas
- cadastrar eventos
- ativar eventos
- comprar ingressos
- cancelar ingressos
- listar dados

```kotlin
fun main() {
    while (true) {
        println("\n--- DENDE EVENTOS (MARSELHA) ---")
        println("1. Inativar Usuário")
        println("2. Reativar Usuário")
        println("3. Listar Usuários")
        println("0. Sair")

        val opcao = readInt("Escolha: ", "Opção inválida!", 0..3)

        when (opcao) {
            1 -> inativarUsuario()
            2 -> reativarUsuario()
            3 -> printTable("TODOS OS USUARIOS", Repositorio.usuarios)
            0 -> {
                println("Encerrando...")
                break
            }
        }
    }
}
```

CONCLUSÃO

- Sistema funcional e organizado
- Utiliza boas práticas de modelagem
- Permite controle de eventos e usuários
- Pode ser melhorado para seguir execução linear no `main`